Chapter 2

Pattern Representation

Assoc. Prof. Dr. Duong Tuan Anh
Faculty of Computer Science and Engineering,
HCMC Univ. of Technology
3/2015

Outline

- 1. Data structures for pattern representation
- 2. Proximity measures
- 3. Size of patterns
- 4. Abstractions of the data set
- 5. Feature extraction
  - Principal component analysis (PCA)
- 6. Feature selection
- 7. Evaluation of classifiers

Pattern representation

- A *pattern* is a physical object or an abstraction notion.
- Depending on the classification problem, *distinguishing features* of the patterns are used. These features are called *attributes*.
- A pattern is the representation of an object by the values taken by the attributes.
- The choice of attributes and *representation* of patterns is very important step in pattern classification.
- A good representation is one which make use of *discriminating attributes* and also reduces the computational burden in pattern classification.

1. Data structures for pattern representation

Patterns as vectors

- Each element of the vector can represent one *attribute* of the pattern.
- Example: Spherical objects, (30, 1) represents a spherical object with 30 units of weight and 1 unit diameter.

A set of patterns.

1.0, 1.0, 1	1.0, 2.0, 1	2.0, 1.0, 1
2.0, 2.0, 1	4.0, 1.0, 2	5.0, 1.0, 2
4.0, 2.0, 2	5.0, 2.0, 2	1.0, 4.0, 2
1.0, 5.0, 2	2.0, 4.0, 2	2.0, 5.0, 2
4.0, 4.0, 1	5.0, 5.0, 1	4.0, 5.0, 1
5.0, 4.0, 1		

The third element gives the *class* of the pattern.

Patterns as strings

- The ***string*** may be viewed as a sentence in a language.

Example 1: a DNA sequence or protein sequence.

- A gene can be defined as a region of the chromosomal DNA constructed with 4 nitrogenous bases: adeline, guanine, cytosine and thymine, which are referred to by A, G, C and T.
- A gene data is arranged in a sequence, such as: GAAGTCCAG…

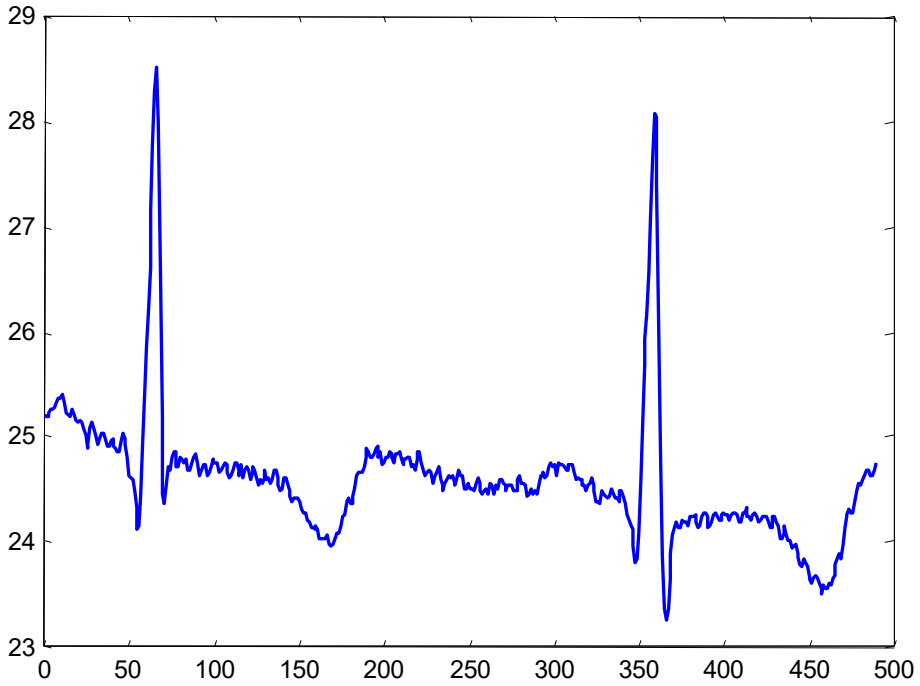
Example 2: (Natural language processing) a sentence in a language can be represented as a string.

"Python is fun"

Pattern as sequence of real values

25.1750
25.2250
25.2500
25.2500
25.2750
25.3250
25.3500
25.3500
25.4000
25.4000
25.3250
25.2250
25.2000
25.1750
..
..
24.6250
24.6750
24.6750
24.6250
24.6250
24.6250
24.6750
24.7500

A *time series* is a sequence of real numbers measured at equal time intervals.



Examples: Financial time series, scientific time series

Figure 2.1 A time series about prices of a stock

Pattern as logical description

- Patterns can be represented as a logical description of the form

  $(x_1 = a_1 \ldots a_2) \wedge (x_2 = b_1 \ldots b_2) \wedge \ldots$

  where x_1 and x_2 are the attributes of the pattern and a_i and b_i are the values taken by the attribute.
- This description consists of a *conjunction* of logical description.
- Example:

  $(color = red \vee white) \wedge (make = leather) \wedge (shape = sphere)$

  to represent a cricket ball.

2. Proximity measures

- In order to classify patterns, they need to be compared against each other and against a standard.
- When a new pattern is presented and we need to classify it, the *proximity* of this pattern to the patterns in the training set is to be found.
- In unsupervised learning, it's required to find some groups in the data so that patterns which are *similar* are put together.
- A number of *similarity* and *dissimilarity measures* can be used.

Distance measure

- A distance measure is used to find the *dissimilarity* between pattern representations. Patterns which are more similar should be closer.
- A distance function could be a *metric* or a *non-metric*.
- A *metric* is a measure for which the following properties hold:

1. *Positive reflexivity*: $d(x,x) = 0$
2. *Symmetric*: $d(x, y) = d(y, x)$
3. *Triangular inequality*: $d(x, y) \leq d(x, z) + d(z, y)$

Distance measure (cont.)

- The popular distance metric called the Minkowski metric is of the form

$$d^m(X,Y) = \left(\sum_{k=1}^{d} | x_k - y_k |^m \right)^{\frac{1}{m}}$$

When m = 1 it is called the ***Manhattan distance*** or L_1 distance.

The most popular is the ***Euclidean distance*** or the L_2 distance when m = 2.

$$d^2(X,Y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \ldots + (x_d - y_d)^2}$$

Distance measure (cont.)

- Example: X = (4, 1, 3) and Y = (2, 5, 1), the Euclidean distance:

$$d^2(X,Y) = \sqrt{(4-2)^2 + (1-5)^2 + (3-1)^2} = 4.9$$

Weighted Distance Measure

The weighted distance metric is of the form

$$d^m(X,Y) = \left(\sum_{k=1}^{d} w_k \times | x_k - y_k |^m \right)^{\frac{1}{m}}$$

where w_k is the weight associated with the k-th attribute (feature)

The mean of a dataset (centroid)

Example:

Given of a dataset consisting of the 5 patterns as follows:

$X_1 = (1, 1)$, $X_2 = (1, 2)$, $X_3 = (2, 1)$, $X_4 = (1.6, 1.4)$, $X_5 = (2, 2)$

Mean = $\langle(1+1+2+1.6+2)/5, (1+2+1+1.4+2)/5\rangle = \langle 1.52, 1.48\rangle$

The mean of the points in dataset C is given by:

$$(1/N_C) \sum X_i \in C$$

Where N_C is the number of the points in the dataset C.

Mahalanobis distance

- Suppose 2 datasets shown as in the figure and the test point (the large 'X'). Let check if the *X* is part of the data. For the figure on the left, the answer is *yes* while for the (right), the answer is *no*.
- We look at not only the *mean* of the dataset but also the *spread* of the actual data points.
- If the data is tightly controlled then the test point has to be close the mean, while if the dataset is very spread out then the distance of the test point from the mean does not matter much.
- The **Mahalanobis distance** is the distance that takes this into account.

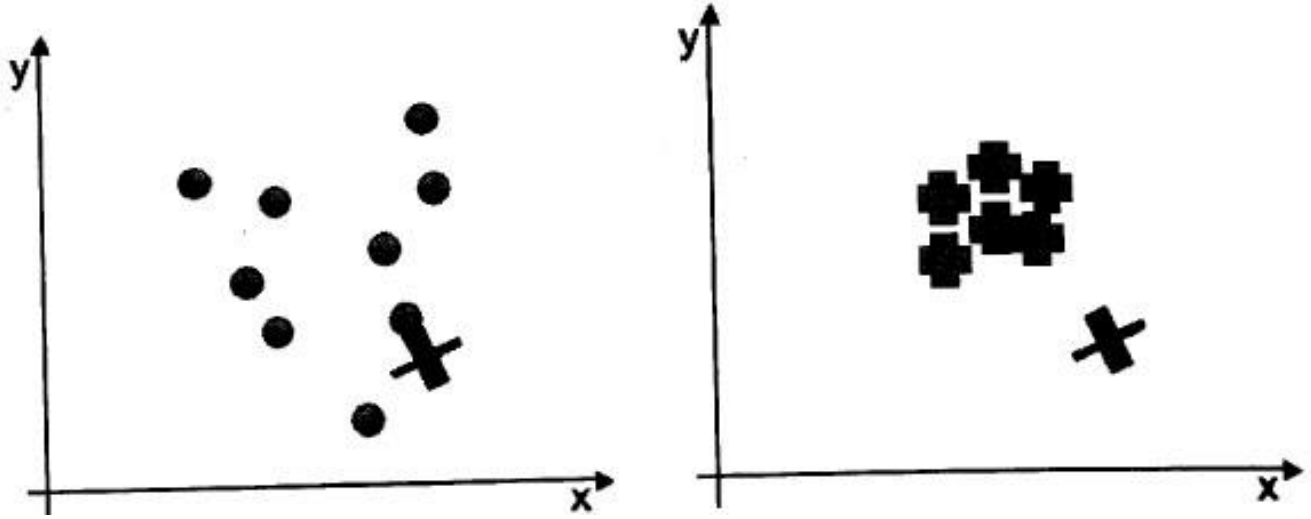


Figure 2.4 Two different datasets and a test point

Mahalanobis distance

The Mahalanobis distance from data point *X* to the mean is given by

$$D_M(X) = \sqrt{(X-\mu)^T \Sigma^{-1} (X-\mu)}$$

where *X* is the column vector of data point with mean vector μ and Σ^{-1} is the inverse of the *covariance matrix* (of the dataset).

• The Mahalanobis distance is a generalization of Euclidean distance. It is an approximate measure when attributes have different scales and are correlated, but still are Gausian distributed.

• If we set the covariance matrix to *identity matrix*, the Mahalanobis distance reduces to the Euclidean distance.

• Computing Mahalanobis distance incurs high computational cost.

Example of Mahalanobis distance

- In a two-class, two-attribute classification, the feature vectors are described by a covariance matrix:

$$\Sigma = \begin{vmatrix} 1.1 & 0.3 \\ 0.3 & 1.9 \end{vmatrix}$$

And the mean vectors are $\mu_1 = [0, 0]^T$, $\mu_2 = [3, 3]^T$, respectively.

Classify the vector $[1.0, 2.2]^T$.

Compute the Mahalanobis distance from the test point to the two means

$$D_m^2(X, \mu_1) = (X - \mu_1)^T \Sigma^{-1} (X - \mu_1) = [1.0, 2.2] \begin{bmatrix} 0.95 & -0.15 \\ -0.15 & 0.55 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.2 \end{bmatrix} = 2.952$$

$$D_m^2(X, \mu_2) = (X - \mu_2)^T \Sigma^{-1} (X - \mu_2) = [-2.0, -0.8] \begin{bmatrix} 0.95 & -0.15 \\ -0.15 & 0.55 \end{bmatrix} \begin{bmatrix} -2.0 \\ -0.8 \end{bmatrix} = 3.672$$

The vector is assigned to class 1, since it is closer to μ_1.

The Mahalanobis distance is widely used in supervised classification techniques and in clustering. A test point is classified as belonging to that class for which the Mahalanobis distance is minimal.

The Mahalanobis distance can also be used to detect *outliers*, as samples that have a significantly greater Mahalanobis distance from the mean than the rest of the samples.

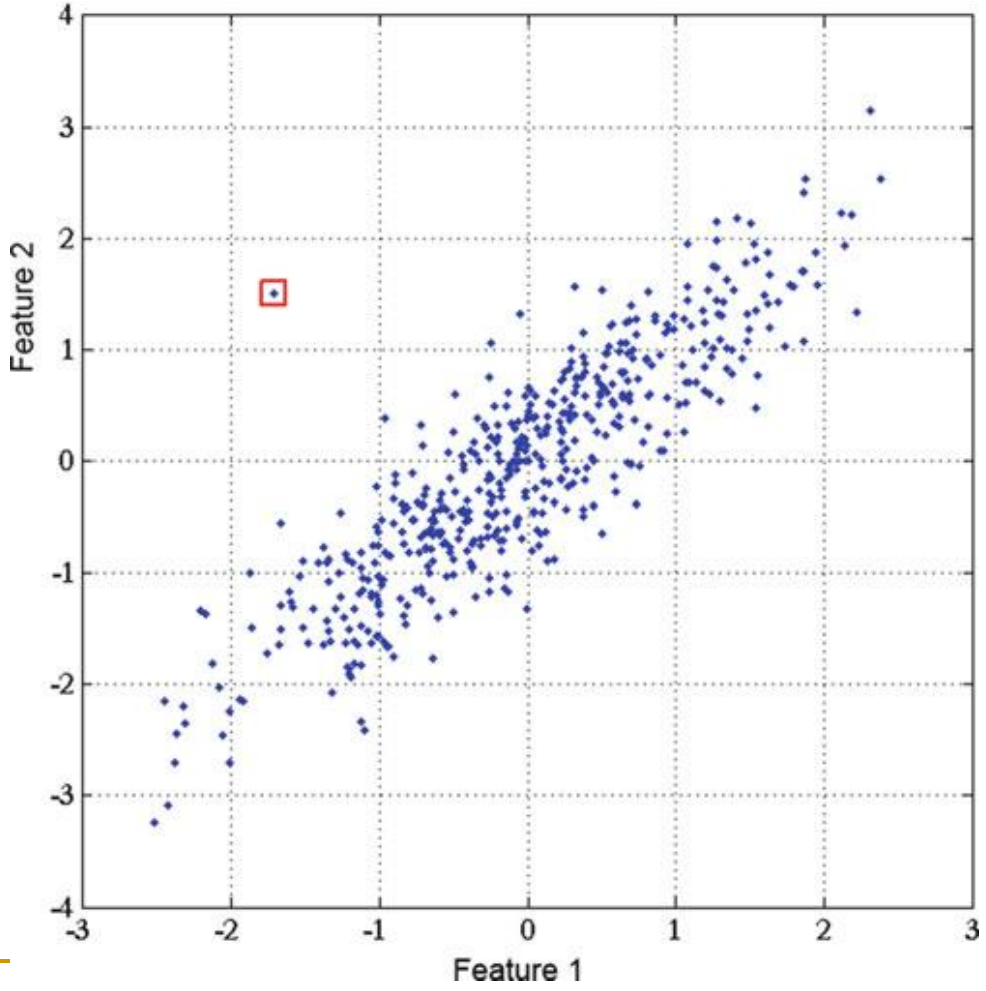


Figure 2.5 Example of an outlier

Non-metric similarity function

- Non-metric similarity function is the similarity function which does not obey either the *triangular inequality* or symmetry.
- This kind of similarity functions are useful for images or string data.
- Example: *k-median distance*

If $X = (x_1, x_2, \ldots, x_d)$ and $Y = (y_1, y_2, \ldots, y_d)$, then

$d(X, Y) = k\text{-median}\{|x_1 - y_1|, \ldots |x_d - y_d|\}$

where the k-median operator returns the *k*-th value of the ordered *difference vector*.

Example: if $X = (50, 3, 100, 29, 62, 140)$ and $Y = (55, 15, 80, 50, 70, 170)$ then Difference vector = {5, 12, 20, 21, 8, 30).

$d(X,Y) = k\text{-median}(5, 8, 12, 20, 21, 30)$.

If $k = 3$ then $d(X, Y) = 12$.

Edit distance (Levenshtein distance)

- ***Edit distance*** measures the distance between two strings.
- The edit distance between two strings s_1 and s_2 is defined as the minimum number of point mutations required to change s_1 to s_2.
- A point mutations can be one of the following operations:
  - Changing a letter
  - Inserting a letter
  - Deleting a letter
- The following recurrence relation defines the edit distance between two strings.
  - $d(\text{“ ”}, \text{“ ”}) = 0$
  - $d(s, \text{“ ”}) = d(\text{“ ”}, s) = ||s||$
  - $d(s_1+ch_1, s_2+ch_2) = \min(d(s_1, s_2)+ \{\text{if } ch_1 = ch_2 \text{ then 0 else 1}\},$
    $d(s_1+ ch_1, s_2)+1, d(s_1, s_2 + ch_2)+1))$

Edit distance (cont.)

- Example:
  - If *s* = “TRAIN” and *t* = “BRAIN”, then edit distance = 1 because this requires a change of just one letter.
  - If *s* = “TRAIN” and *t* = “CRANE”, then edit distance = 3. We can write the edit distance from *s* to *t* to be the edit distance between “TRAI” and “CRAN” + 1 (since N and E are not the same). It would then be the edit distance between “TRA” and “CRA” + 2 (sine I and N are not the same). Proceeding in this way, we get the edit distance to be 3.
- Application: Edit distance can be used in speech recognition
- Dynamic Time Warping (DTW) is also a non-metric distance for time series data.

3. Size of patterns

- The size of a pattern depends on the attributes being considered. In some cases, the length of the patterns may be a variable.
- Example: in document retrieval, documents may be of different lengths.
- Example: in time series data mining, time series may be of different lengths.
- This problem can be handled in different ways.
- *Normalization* of data can make all the patterns have the same dimensionality (i.e. the same number of attributes)

Normalization of data

- ***Normalization*** can also be done to given the same importance to every feature in a data set of patterns.
- Ex: Consider a data set of patterns with two features:

  X_1: (2, 120), X_2: (8, 533), X_3: (1, 987), X_4: (15, 1121), X_5: (18, 1023)
- Normalization givens equal importance to every feature. Dividing every value of the first feature by its maximum value (18) and dividing every value of the second feature (1121) by its maximum value.

  X'_1= (0.11, 0.11), X'_2= (0.44, 0.48), X'_3 = (0.06, 0.88), X'_4= (0.83, 1.0), X'_5= (1.0, 0.91).

- Note: Some similarity measures can deal with unequal lengths. One such similarity measure is the *edit distance*.

4. Abstraction of the data set

In supervised learning, a set of training patterns where the class label for each pattern is given, is used for classification. The *large* training set may not be used because the processing time may be too long but an ***abstraction*** of the training set can be used.

There are some ways of abstractions:

- 1. No abstraction of patterns.
- 2. Single ***representative*** per class.
- 3. Multiple representatives per class

5. Feature extraction

- *Feature extraction*: detecting and isolating various desired features of patterns.
- It is the operation of extracting features for identifying or interpreting meaningful information from the data.
- This is especially relevant in the case of image data where feature extraction involves automatic recognition of various features.
- Feature extraction is an essential *preprocessing* step in pattern recognition.
- One important method of feature extraction:
  - Principal Component Analysis (PCA)

Principal Component Analysis (PCA)

- PCA is a mathematical procedure that transform a number of correlated attributes into a smaller number of non-correlated attributes called *principal components*.
- PCA is a method of *dimensionality reduction*

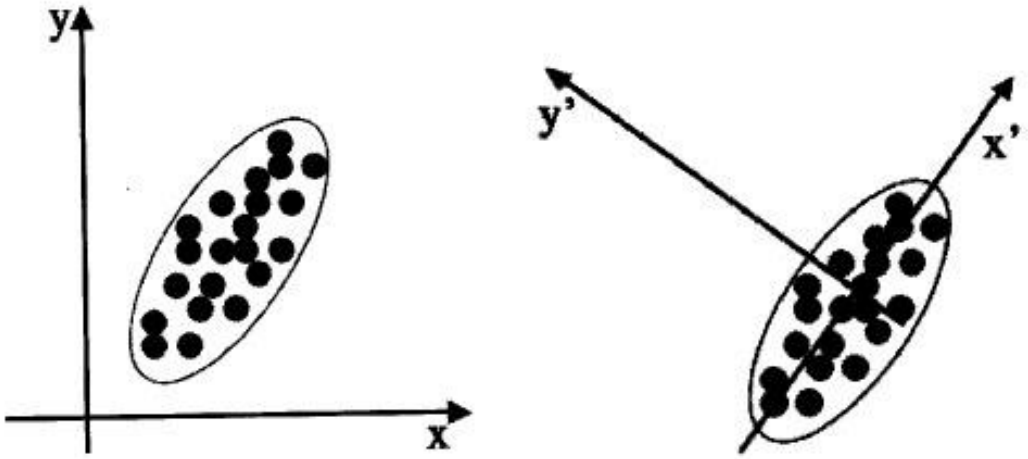


Figure 2.7: Two different sets of coordinate axes. The second consists of a rotation and translation of the first and was found by using PCA.

• The goal of PCA is to identify the *most meaningful basis* to re-express a data set.

Change of basis

- "Is there another basis which is a *linear* combination of the original basis, that best re-express our data set?
- Let ***X*** be the original data set, where each column is a single sample of our data set. ***X*** is an $m \times n$ matrix. Let ***Y*** be another $m \times n$ matrix related by a linear transformation ***P***. So ***X*** is the original recorded data and ***Y*** is a new representation of that data set.

$$\boldsymbol{PX} = \boldsymbol{Y} \qquad (1)$$

- Equation (1) represent a change of basis.
  - ***P*** is a matrix that transforms ***X*** into ***Y***.
  - Geometrically, ***P*** is a rotation and a translation.
  - The rows of ***P***, $\{p_1, \ldots, p_m\}$, are set of new basis vectors for expressing the columns of ***X***.

$$PX = \begin{bmatrix} p_1 \\ .. \\ p_m \end{bmatrix} \begin{bmatrix} x_1 & .. & x_n \end{bmatrix}$$

$$Y = \begin{bmatrix} p_1.x_1 & .. & . & p_1.x_n \\ . & . & . & . \\ . & . & . & . \\ p_m.x_1 & . & . & p_m.x_n \end{bmatrix}$$

Each column of ***Y***:

$$y_i = \begin{bmatrix} p_1.x_i \\ . \\ . \\ p_m.x_i \end{bmatrix}$$

$\underline{\boldsymbol{y}_i}$ is a projection of $\underline{\boldsymbol{x}_i}$ on to the basis of $\{p_1, \ldots, p_m\}$.

Now, several questions arise:

- What is the best way to re-express ***X***?
- What is a good choice of basis ***P***?

- From the matrix $\boldsymbol{X}$, let build the covariance matrix of $\boldsymbol{X}$, called $\boldsymbol{C_X}$.
  - $\boldsymbol{C_X}$ is a square symmetric $m \times m$ matrix
  - The diagonal terms of $\boldsymbol{C_X}$ are the variances of particular attributes.
  - The off-diagonal terms of $\boldsymbol{C_X}$ are the covariance between two corresponding attributes.
- $\boldsymbol{C_X}$ captures the noise and redundancy in our data set.
  - In the *diagonal* terms, *large values correspond to interesting structure.*
  - In the *off-diagonal* terms, **large values correspond to high redundancy.**
- Now, the question is: **What features do we want to have in C_Y?**
- What we want the optimized covariance matrix $\boldsymbol{C_Y}$ look like?

Diagonalize the covariance matrix

- What we want the optimized covariance matrix C_Y look like?
  - All off-diagonal terms in C_Y should be zero. Thus, C_Y must be a ***diagonal matrix***. ***Y*** is ***decorrelated***.
  - Each successive dimension in ***Y*** should be rank-ordered according to variance.
- There are many methods for ***diagonalizing*** C_Y. PCA selects the easiest method: PCA assumes that ***P*** is an orthonormal matrix.
- ***P*** acts as a generalized rotation to align a basis with the axis of maximum variance.
- In multiple dimensions this could be performed by a simple algorithm:

PCA algorithm

- 1. Select *a normalized direction* in m-dimensional space along which the variance in $\boldsymbol{X}$ is maximized. Save this vector as $\boldsymbol{p}_1$.
- 2. Find another direction along which variance is maximized, however, because of the orthonormality condition, restrict the search to all directions *orthogonal to* all previous selected directions. Save this vector as $\boldsymbol{p}_i$.
- 3. Repeat this procedure until *m* vectors are selected.

The resulting ordered set of $\boldsymbol{p}$'s are the *principal components*.

Summary of Assumptions in PCA

1. Linearity
2. **Large variances have important structure**
3. *The principal components are orthogonal*

Solving PCA using eigenvector decomposition

- Given the data set $\boldsymbol{X}$, an $m \times n$ matrix, where m is the number of attributes and n is the number of samples. The goal is as follows.

  "Find some orthonormal matrix $\boldsymbol{P}$ in $\boldsymbol{Y} = \boldsymbol{PX}$ such that $\boldsymbol{C_Y} = (1/n)\boldsymbol{YY}^T$ is a diagonal matrix. The rows of $\boldsymbol{P}$ are the principal components of $\boldsymbol{X}$."

- We begin by rewriting $\boldsymbol{C_Y}$ in terms of the unknown variable.

$$\begin{aligned} \boldsymbol{C_Y} &= (1/n)\boldsymbol{YY}^T \\ &= (1/n)(\boldsymbol{PX})(\boldsymbol{PX})^T \\ &= (1/n)\boldsymbol{PXX}^T\boldsymbol{P}^T \\ &= \mathbf{P}(1/n)\boldsymbol{XX}^T\boldsymbol{P}^T \\ \boldsymbol{C_Y} &= \boldsymbol{PC_XP}^T \end{aligned}$$

Note that the covariance matrix $\boldsymbol{C_X}$ appears in the last line. $\boldsymbol{C_X}$ is a symmetric matrix.

- **Theorem**: Let $\boldsymbol{A}$ be a square $n \times n$ symmetric matrix with associated eigenvectors $\{\boldsymbol{e_1}, \boldsymbol{e_2}, \ldots, \boldsymbol{e_n}\}$. Let $\boldsymbol{E} = [\boldsymbol{e_1}\ \boldsymbol{e_2}\ \ldots \boldsymbol{e_n}]$ where ith column of $\boldsymbol{E}$ is the eigen vector $\boldsymbol{e_i}$. There exists a diagonal matrix $\boldsymbol{D}$ such that $\boldsymbol{A} = \boldsymbol{EDE}^T$.
- Now come the trick. We **select the matrix P to be the matrix where each row p_i is an eigenvector of C_X**. By this selection, $\mathbf{P} = \boldsymbol{E}^T$. With this relation, we can finish evaluating $\boldsymbol{C_Y}$.

$$\begin{aligned} \boldsymbol{C_Y} &= \boldsymbol{PC_XP}^T \\ &= \boldsymbol{P}(\boldsymbol{EDE}^T)\boldsymbol{P}^T \\ &= \boldsymbol{P}(\boldsymbol{P}^T\boldsymbol{DP})\boldsymbol{P}^T \\ &= (\boldsymbol{PP}^T)\boldsymbol{D}(\boldsymbol{PP}^T) \end{aligned}$$

- Since $\boldsymbol{P}^T = \boldsymbol{P}^{-1}$ (Theorem: the inverse of an orthogonal matrix is its transpose), we get

$$\begin{aligned} \boldsymbol{C_Y} &= (\boldsymbol{PP}^{-1})\boldsymbol{D}(\boldsymbol{PP}^{-1}) \\ &= \boldsymbol{D} \end{aligned}$$

- It is evident that the choice of $\boldsymbol{P}$ diagonalizes $\boldsymbol{C_Y}$. This was the goal of PCA.

Computing PCA

- We can summarize the results of PCA in the matrix $\boldsymbol{P}$ and $\boldsymbol{C_Y}$.
  - The principal components of $\boldsymbol{X}$ are the eigenvectors of $\boldsymbol{C_X}$, the covariance matrix of $\boldsymbol{X}$.
  - The ith diagonal value of $\mathbf{C_Y}$ is the variance of $\boldsymbol{X}$ along $\boldsymbol{p_i}$.
- In practice computing PCA of a dataset $\boldsymbol{X}$ consists of:
  - Subtracting off the mean of each attribute in $\boldsymbol{X}$.
  - Computing the covariance matrix $\boldsymbol{C_X}$. ($\boldsymbol{C_X} = (1/n).\boldsymbol{X}.\boldsymbol{X}^T$)
  - Computing the eigenvalues and eigenvectors of $\boldsymbol{C_X}$.
  - Reject the eigenvalues less than some threshold, leaving l dimensions in the data.
  - If $\boldsymbol{P}$ is the matrix consisting of eigenvectors of $\boldsymbol{C_X}$ as the row vectors, we get $\boldsymbol{y_i} = \boldsymbol{P}(\boldsymbol{x_i} - \text{mean}(\mathbf{X}))$ $\quad i = 1 \ldots n$

Example (PCA)

- A data set contains four patterns in two dimension: (1,1), (1, 2), (4,4) and (5,4). With these four patterns, the mean vector is

  [2.75 2.75].

- With these four patterns, the covariance matrix is:

$$C_X = \begin{bmatrix} 4.25 & 2.92 \\ 2.92 & 2.25 \end{bmatrix}$$

The eigenvalues of C_X are λ_1 = 6.336 and λ_2 = 0.1635.

Since the second eigenvalue is very small compared to the first eigenvalue, the second eigenvector can be left out. The eigenvector which is most significant is computed as

$$e_1 = \begin{bmatrix} 0.814 \\ 0.581 \end{bmatrix}$$

Example (PCA) (cont.)

- To transform the patterns on to the eigenvector, the pattern (1, 1) gets transformed to

$$e_1.x_1 = \begin{bmatrix} 0.814 & 0.581 \end{bmatrix} \times \begin{bmatrix} 1-2.75 \\ 1-2.75 \end{bmatrix} = \begin{bmatrix} 0.814 & 0.581 \end{bmatrix} \times \begin{bmatrix} -1.75 \\ -1.75 \end{bmatrix} = -2.44$$

Similarly, the patterns (1, 2), (4, 4) and (5, 4) get transformed to -1.86, 1.74 and 2.56.

When we try to get the original data using the transformed data, some information is lost. For pattern (1, 1), using the transformed data, we get

$$\begin{bmatrix} 0.814 & 0.581 \end{bmatrix}^T \times (-2.44) + \begin{bmatrix} 2.75 \\ 2.75 \end{bmatrix} = \begin{bmatrix} -1.99 & -1.418 \end{bmatrix}^T + \begin{bmatrix} 2.75 \\ 2.75 \end{bmatrix} = \begin{bmatrix} 0.76 \\ 1.332 \end{bmatrix}$$

PCA tool

- PCA tool is available in:
  - MATLAB
  - The data mining software Weka (Attribute selection – “filters”)
  - Scikit-learn

6. Feature selection

- Features used for classification may be always be meaningful. Removal of some of the features may give a better classification accuracy.
- Features which are useless for classification are found and left out.
- Feature selection can:
  - Speed up the process of classification
  - Ensure that the classification accuracy is optimal.
- Feature selection algorithms involve searching through different feature subsets. Feature selection algorithms = *search algorithms*.

- Feature selection can ensure improvement of classification accuracy, due to 2 reasons:
  - 1. The performance of a classifier depends on (i) sample size, (ii) number of features and (iii) classifier. The probability of misclassification does not increase as the number of features increases, as long as the number of training patterns is large. If the number of training patterns is small, adding features may degrade the accuracy of a classifier.

    $\Rightarrow$ *Peaking phenomenon* or *the curse of dimensionality*

2. As dimensionality increases, the distance to the nearest data point approaches the distance of the furthest data point. This effects the results of the nearest neighbor problem.

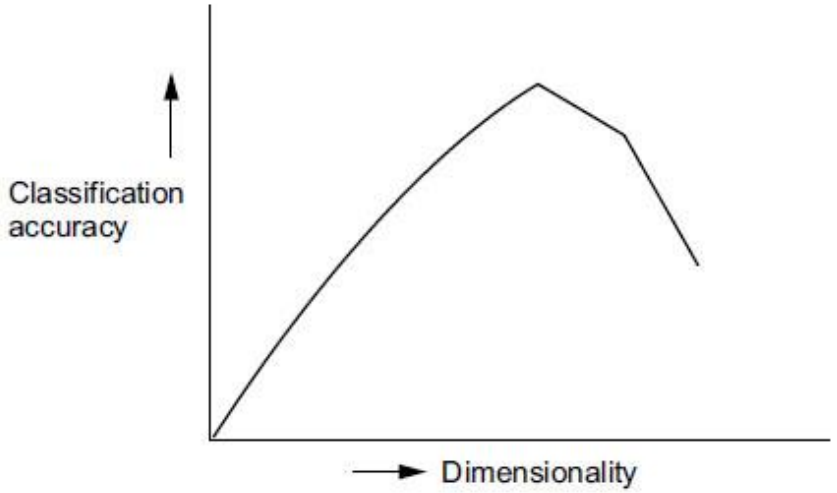


Figure 2.8 The curse of dimensionality

Exhaustive search

- The most straightforward approach to the problem of feature subset selection is *exhaustive search*.
- Search through all the feature subsets and find the best subset.
- If the patterns consist of *d* features and a subset of size *m* features is to be found with the smallest classification error, it entails searching all $C(m,d)$ possible sub-sets of size *m* and selecting the subset with the highest criterion function $J(.)$, where $J = (1- P_e)$.

  P_e : probability of classification error
- This technique is impractical to use even for moderate value of *d* and *m*. Even when *d* is 24 and *m* is 12, approximately 2.7 million feature subsets must be checked.

Note: $C(m,d) = d!/[m!(d – m)!]$

Sequential selection

- These methods operate either by evaluating growing feature sets or shrinking feature sets.
- The method that starts with the empty set and add features one after the other is called *sequential forward selection* (SFS). This method is bottom-up.
- The method that starts with the full set and delete features one after the other is called *sequential backward selection* (SBS). This method is top-down.
- These methods are just heuristics. They do not guarantee the optimal result.
- The stepwise forward selection and backward selection methods suffer the "*nesting effect*".
- In the case of top-down search the discarded features can not be re-selected while in the case of the "bottom-up" search the features once selected can not be later discarded.

Sequential selection (cont.)

The features are selected according to their significance. The most significant feature is selected to be added to the feature subset at every stage.

The most significant feature is the feature that give the highest increase in criterion function value as compared to that of the others before adding the feature.

If U_0 is the feature added to the set X_k consisting of k features then the significance of this is:

$$S_{k+1}(U_0) = J(X_k \cup U_0) - J(X_k)$$

i.e. $$J(X_k \cup U_0^r) = \max_{0 \le i \le \phi} J(X_k \cup U_0^i)$$

where ϕ is the number of all the possible U_0.

Sequential selection (cont.)

- The least significant feature with respect to set X_k is

$$J(X_k \cup U_0^r) = \min_{0 \le i \le \phi} J(X_k \cup U_0^i)$$

Combination of *forward selection* and *backward selection*: The stepwise forward selection and backward selection methods can be combined so that, at each step, the procedure selects the best attribute and removes the worst from the remaining attributes.
This method is called ***sequential floating search***.

Sequential floating forward search

The principle of SFFS is as follows:

- Step 1: Let $k = 0$
- Step 2: Add the most significant feature to the current subset of size k. Let $k = k+1$
- Step 3: Conditionally remove the least significant feature from the current subset.
- Step 4: If the current subset is the best subset of size k-1 found so far, let $k = k$-1 and go to step 3. Else return the conditionally removed feature and go to Step 2.

Note: The algorithm starts with no feature and then adds features one by one.

Sequential floating backward search

The method is similar to the SFFS except that the backward search is carried out first and then the forward search. The principle of SFBS is as follows:

- Step 1: Let $k = 0$
- Step 2: Remove the least significant feature to the current subset of size $d - k$. Let $k = k + 1$
- Step 3: Conditionally add the most significant feature not in the current subset.
- Step 4: If the current subset is the best subset of size $d - (k - 1)$ found so far, let $k = k - 1$ and go to step 3. Else remove the conditionally added feature and go to Step 2.

Note: The algorithm starts with all the features (size = d) and then removes them one by one.

Stochastic Search Techniques

- **Genetic algorithm** is a stochastic search technique which is often used for feature selection.
- The *population* in the GA consists of strings which is binary.
- Each string (*chromosome*) is of length *d*, with each position *i* being zero or one depending on the absence or presence of feature *i* in the set.
- Each feature subset is coded as a *d*-element bit string. Each string in the population is a feature selection vector α, where $\alpha = \alpha_1, \ldots, \alpha_d$ and α_i assumes a value 0 if the *i*th feature is excluded and 1 if it is present in the subset.
- To compute the *fitness* of a chromosome, it is evaluated by determine its performance on the training set.
- Besides genetic algorithm, **Tabu search** and **simulate annealing** can be used in feature selection.

Filters and wrappers

- There are a number of different approaches to feature selection. In general, they are divided into two kinds: *filters* and *wrappers*.
- In ***filter*** model, the features are filtered independent of the classification algorithm.
  - The criterion function used in the filter is not based on *classification error*. It is based on general characteristics of the data.
- In ***wrapper*** model, the features subset selection algorithm conducts a search for a good subset using the classification algorithm itself as part of the evaluation function.

Filters and wrappers (cont.)

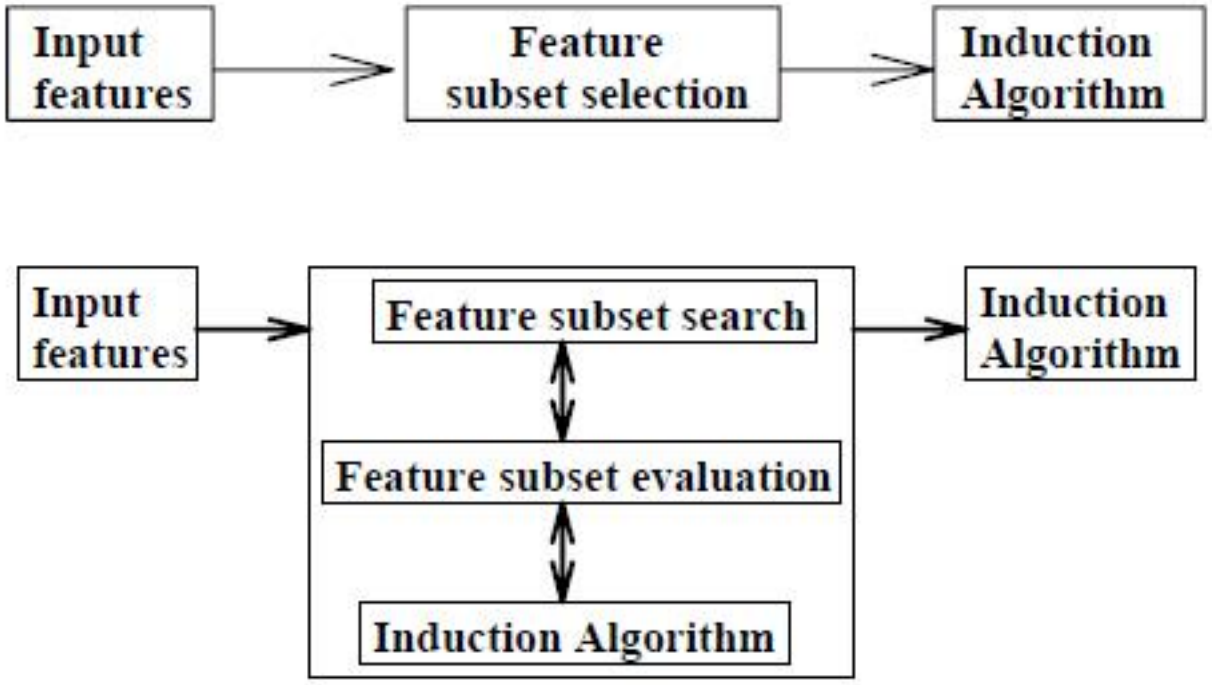


Figure 2.9: (a) Filter model, (b) Wrapper model

Feature extraction: feature selection ($F' \subset F$) and feature abstraction ($F' \not\subset F$)

7. Classifier Accuracy Measures

- Accuracy is better measured on a *test set* consisting of class-labeled tuples that were not used to train the model.
- The accuracy of a classifier on a given test set is the percentage of test set tuples that are correctly classified by the classifier. It is also called the *overall recognition rate* of the classifier.
- The **error rate** or **misclassification rate** of a classifier, *M*, is simply 1- *Acc*(*M*), where *Acc*(*M*) is the accuracy of *M*.

- The *confusion matrix* is a useful tool for analyzing how well the classifier can recognize tuples of different classes. Give *m* class, a confusion matrix is a table of at least size *m* by *m*.
- An entry, CM_{ij} indicates the number of tuples of class *i* that were labeled by the classifier as class *j*.
- For a classifier to have good accuracy, most of the tuples would be represented along the diagonal of the confusion matrix, from entry CM_{11} to CM_{mm}, with the rest of the entries being close to zero.

		Predicted class C_1	Predicted class C_2
Actual class	C_1	true positives	false negatives
	C_2	false positives	true negatives

The *sensitivity* and *specificity* measures can be used.

Sensitivity is also referred to as the *true positive* (*recognition*) *rate* (that is, the proportion of positive tuples that are correctly identified), while ***specificity*** is the *true negative* (*recognition*) *rate* (that is, the proportion of negative tuples that are correctly identified).

$$sensitivity = \frac{t\_pos}{pos} \qquad specificity = \frac{t\_neg}{neg}$$

where *t_pos* is the number of true positive tuples, *pos* is the number of positive tuples, *t_neg* is the number of true negative tuples and *neg* is the number of negative tuples.

We may use ***precision*** to access the percentage of tuples labeled as "positive" that actually are positive tuples.

$$precision = \frac{t\_pos}{t\_pos + f\_pos}$$

- ***Accuracy*** is a function of sensitivity and specificity.

$$accuracy = sensitivity \frac{pos}{(pos + neg)} + specificity \frac{neg}{(pos + neg)}$$

$$= \frac{t\_pos + t\_neg}{pos + neg}$$

Example

Given a confusion matrix:

		Predicted class C_1	Predicted class C_2
Actual class	C_1	4	1
	C_2	2	4

Sensitivity = 4/5

Specificity = 4/6

Precision = 4/(4+2) = 4/6

Accuracy = (4+4)/(5+6) = 8/11

8. Evaluating the accuracy of a classifier

To estimate how good a classifier is, an estimate can be made using the training set. Assume that the training data is a good representative of the data.

Usually the training set is divided into smaller subsets. One of the subsets is used for *training* while the other is used for *testing*.

Different methods of testing are as follows.

- **Holdout method**: The training set is divided into two subsets. Typically *two-thirds* of the data is used for training and *one-third* is used for testing. It could also be one-haft for training and one-haft for testing or any other proportion.

- **Random sub-sampling**: In this method, the holdout method is repeated a number of times with different training and test data each time.

Random subsampling

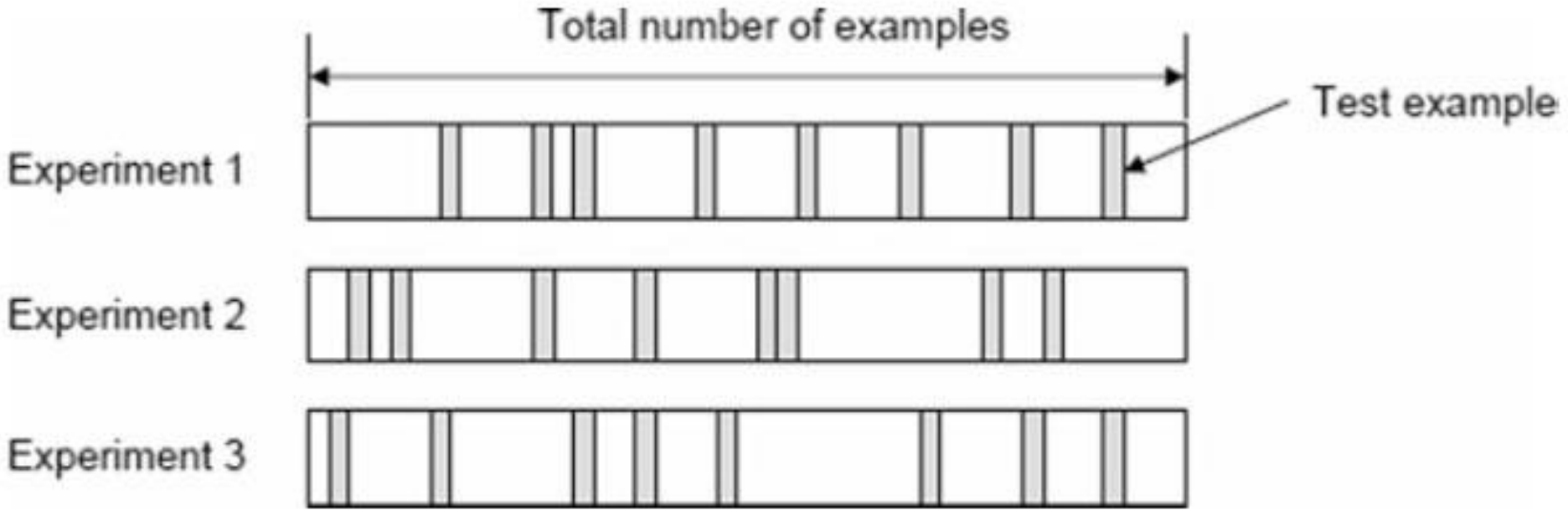


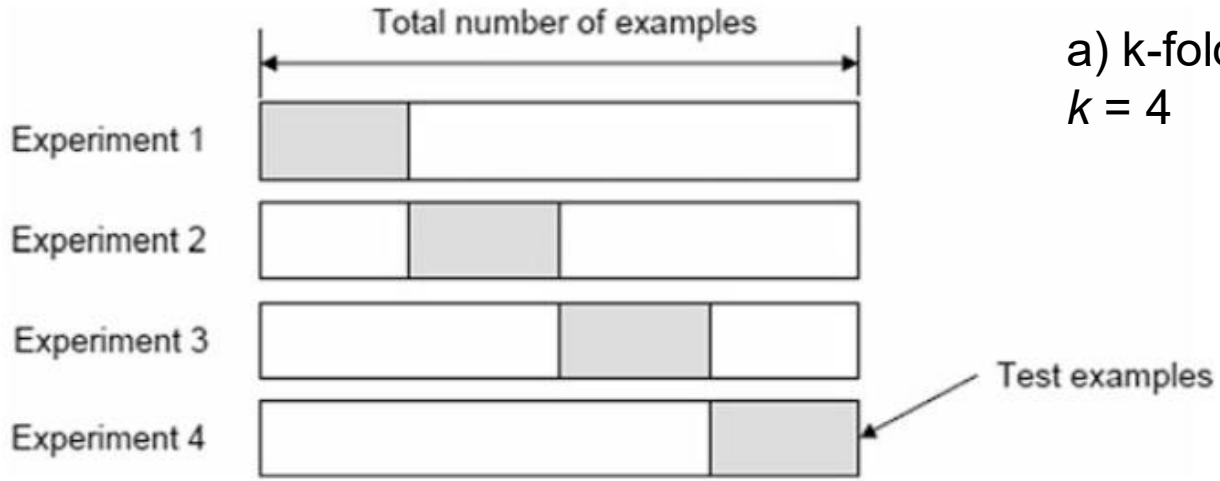
Figure 2.10 Random subsampling

- If the process is repeated *k* times, the overall accuracy will be

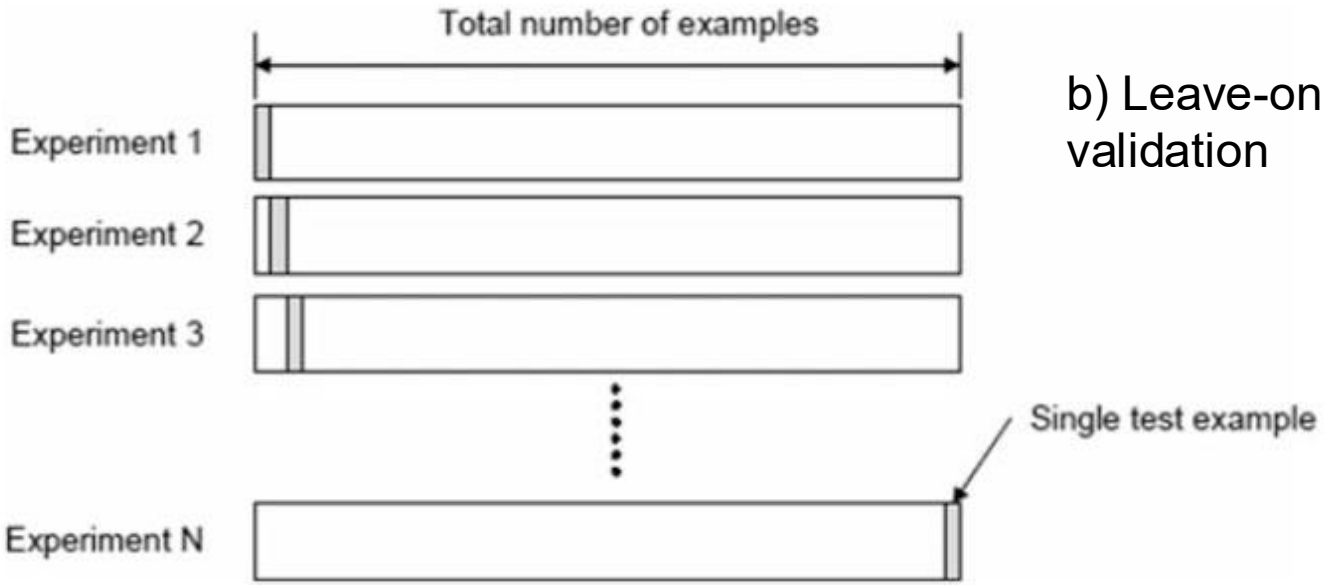
$$acc_{overall} = \frac{1}{k}\sum_{i=1}^{k} acc_i$$

Cross validation

- **Cross-validation**: In cross-validation, each pattern is used the same number of times for training and exactly once for testing. An example of cross-validation is the ***two-fold cross validation***. Here the data set is divided into two equal subsets. First, one set is used for training and the other for testing. Then the roles of the subsets are swapped – the set for training becomes the set for validation and vice versa.
- In ***k-fold cross-validation***, the data is divided into *k* equal subsets. During each run, one subset is used for testing and the rest of the subsets are used for training. The procedure is carried out *k* times so that each subset is used for testing once.
  - A special case of the *k*-fold cross-validation is when $k = n$, where *n* is the number of patterns in the data set. This is called the ***leave-one-out*** approach, where each test set contains only one pattern. The procedure is repeated *n* times.



a) k-fold cross validation, with $k = 4$



b) Leave-one-out cross-validation

Figure 2.11 k-Fold cross-validation and Leave-one-out cross-validation

Bootstrap

- The dataset is *sampled with replacement*, i.e., an instance already chosen for training is put back into the original dataset and can be chosen again so that there can be duplicate objects in the training set. The **remaining instances not selected for training are used for testing**.
- The process is repeated for a specified number of folds, *k*.
- In the bootstrap, we **sample *N* instances from a dataset of size *N* with replacement.** The original dataset is used to form the training set and the test set for cross-validation. The probability that we pick an instance is 1/*N*; and the probability that it is not picked is 1-1/*N*. The probability that we do not pick it after *N* draws is

$$(1 - 1/N)^N \approx e^{-1} = 0.368$$

This means that the *training data* contain 63.2% of the instances and the *test data* contain 36.8%.

Boostrap (cont.)

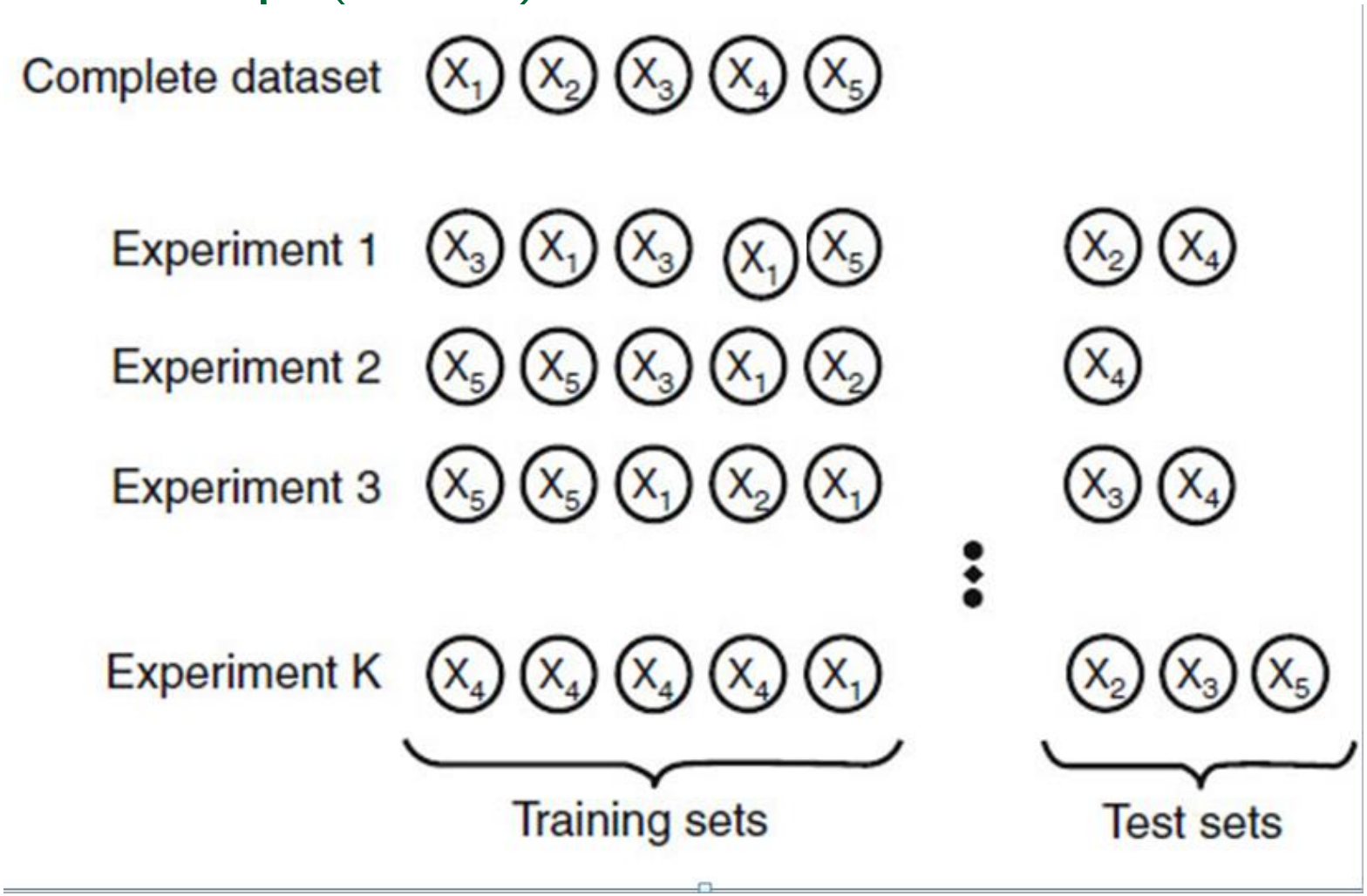


Figure 2.12 The boostrap method

Appendix

- Expectation
- Variance
- Covariance
- The eigenvalue problem

Expectation

- *Expectation*, *expected value*, or *mean* of a random variable *X*, denoted by *E*[*X*], is the average value of *X* in a large number of experiments.

$$E[X] = \sum_i x_i P(x_i) \quad if\ X\ is\ discrete$$

$$E[X] = \int xp(x)dx \quad if\ X\ is\ continuous$$

It has the following properties ($a, b \in \Re$):

$E[aX + b] = aE[X] + b$

$E[X + Y] = E[X] + E[Y]$

Mean is the first moment and is denoted by μ.

Variance

- Variance measures how much X varies around the expected value. If $\mu \equiv E[X]$, the variance is defined as

  $$Var(X) = E[(X - \mu)^2] = E[X^2] - \mu^2$$

- Variance is the second moment minus the square of the first moment.
- Variance, denoted by σ^2, satisfies the following properties (a, $b \in \Re$):

  $$Var(aX + b) = a^2 Var(X)$$

- Square root of $Var(X)$ is called *standard deviation* and is denoted by σ.

Example:

Table: Probability distribution for Y

y	$p(y)$
0	1/8
1	1/4
2	3/8
3	1/4

$\mu = E(Y) = \sum yp(y) = (0)(1/8) + (1)(1/4) + (2)(3/8) + (3)(1/4) = 1.75$

$\sigma^2 = E[(Y - \mu)^2] = \sum (y - \mu)^2 p(y)$

$= (0-1.75)^2(1/8) + (1-1.75)^2(1/4) + (2 -1.75)^2(3/8) + (3-1.75)^2(1/4)$

$= 0.9375$

$\sigma = 0.97$

Covariance

- *Covariance* indicates the relationship between two random variables. If the occurrence of *X* makes *Y* more likely to occur, then the covariance is positive; it is negative if *X*'s occurrence make *Y* less likely to happen and is 0 if there is no dependence.

- $\mathrm{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)] = E[XY] - \mu_X\mu_Y$
  where $\mu_X \equiv E[X]$ and $\mu_y \equiv E[Y]$.

- Some other properties:
  $Cov(X, Y) = Cov(Y, X)$
  $Cov(X, X) = Var(X)$
  $Cov(X + Z, Y) = Cov(X, Y) + Cov(Z, Y)$

Covariance matrix

- Covariance can be used to look at the relationship between all pair of variables within a set of data. We need to compute the covariance of each pair and these are put together into a covariance matrix.

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1d} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2d} \\ . & . & . & . \\ \sigma_{d1} & \sigma_{d2} & \cdots & \sigma_d^2 \end{bmatrix}$$

where σ_{ij} is the covariance of two variables X_i and X_j, i.e.

$\sigma_{ij} = Cov(X_i, X_j) = E(X_i - \mu_i)(X_j - \mu_j) = E(X_iX_j) - \mu_i \mu_j$

Covariance matrix is symmetric since $\sigma_{ij} = \sigma_{ji}$ and $\sigma_{ii} = \sigma_i^2$

If $\mu_i = 0 \ \forall i$ then $\sigma_{ij} = E(X_iX_j)$. Hence, we can compute:

$$\Sigma = (1/n)\boldsymbol{X}.\boldsymbol{X}^T$$

Example of covariance matrix

- Five measurements (observations) each are taken of 3 features, X_1, X_2 and X_3 so that the feature vector is

X_1	X_2	X_3
4.0	2.0	0.6
4.2	2.1	0.59
3.9	2.0	0.58
4.3	2.1	0.62
4.1	2.2	0.63

The mean values are given by μ = [4.1 2.08 0.604]. And the covariance matrix by:

$$\Sigma = \begin{bmatrix} 0.025 & 0.0075 & 0.00175 \\ 0.0075 & 0.0070 & 0.00135 \\ 0.00175 & 0.00135 & 0.0043 \end{bmatrix}$$

where 0.025 is the variance of X_1, 0.0075 is the covariance between X_1 and X_2, 0.00175 is the covariance between X_1 and X_3, etc.

The eigenvalue problem

- Let A be an $n \times n$ matrix, v an $n \times 1$ column vector and λ a scalar. If

$$Av = \lambda v$$

we say that v is an *eigen vector* of A and that λ is an *eigenvalue* of A.

- The characteristic polynomial

The *characteristic polynomial* of a square $n \times n$ matrix A is

$$\det |A - \lambda I|$$

where λ is an unknown variable and I is the $n \times n$ *identity matrix*.

The eigenvalue problem (cont.)

- The Cayley-Hamilton Theorem

In practical calculation, we set the *characteristic polynomial* equal to zero, given the *characteristic equation*:

$$\det |A - \lambda I| = 0$$

The *zeros* of the characteristic polynomial, which are *solutions* to this equation, are the *eigenvalues* of the matrix A.

The second step in solving the eigenvalue problem is to find the *eigen vectors* that correspond to each eigenvalue found in the solution of the characteristic equation.

Example

$$A = \begin{bmatrix} 5 & 2 \\ 9 & 2 \end{bmatrix}$$

To find the eigenvalues, we solve

$$\det |A - \lambda I| = 0$$

where

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\lambda I = \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}$$

The characteristic polynomial in this case is

$$\det | A - \lambda I | = \det \left\| \begin{bmatrix} 5 & 2 \\ 9 & 2 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} \right\| = \det \begin{vmatrix} 5-\lambda & 2 \\ 9 & 2-\lambda \end{vmatrix}$$

$$= (5 - \lambda)(2 - \lambda) - 18 = \lambda^2 - 7\lambda - 8 = (\lambda - 8)(\lambda + 1) = 0$$

The solutions of this equation are the two eigenvalues: $\lambda_1 = 8$, $\lambda_2 = -1$

Now we consider each eigenvalue in turn. An eigen vector of a 2×2 matrix is going to be a column vector with 2 components. If we call these two unknowns *x* and *y*, then we can write the vector as

$$v = \begin{bmatrix} x \\ y \end{bmatrix}$$

For the first eigenvalue ($\lambda_1 = 8$), the eigen vector equation is $Av = \lambda_1 v$

$$Av = \begin{bmatrix} 5 & 2 \\ 9 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 8 \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} 5x + 2y \\ 9x + 2y \end{bmatrix} = 8 \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 8x \\ 8y \end{bmatrix}$$

We have two equations:

$$5x + 2y = 8x \quad (1)$$

$$9x + 2y = 8y \quad (2)$$

From (1) and (2), we get: $y = 3x/2$. The system has only one free variable. So we choose $x = 2$, then $y = 3$. Then the eigen vector corresponding to $\lambda_1 = 8$ is:

$$v_1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Now we check this result

$$Av_1 = \begin{bmatrix} 5 & 2 \\ 9 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 5.2 + 2.3 \\ 9.2 + 2.3 \end{bmatrix} = \begin{bmatrix} 16 \\ 24 \end{bmatrix} = 8 \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

So, the result satisfies: $Av_1 = \lambda_1 v_1$

For the second eigenvalue: $\lambda_2 = -1$

We do the same procedure and obtain the second eigenvector:

$$v_2 = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$$

Summarizing, we have found the eigenvectors of the matrix A to be

$$v_1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \qquad v_2 = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$$

References

- M. N. Murty and V. S. Devi, 2011, *Pattern Recognition – An Algorithmic Approach*, Springer.
- G. Dougerty, 2013, *Pattern Recognition and Classification – An Introduction*, Springer
- E. Alpaydin, 2010, *Introduction to Machine Learning*, The MIT Press.
- J. Shlens, 2014, *A Tutorial on Principal Component Analysis*.
- S. Theodoridis, K. Koutroumbas, 2009, *Pattern Recognition – 4th Edition*, Academic Press.

Terminology

- distance measure*: độ đo khoảng cách*, similarity measure: *độ đo tương tự*, edit distance: *độ đo biên tập*, abstraction of a data set: *trích yếu một tập dữ liệu*, linear discriminant analysis: *phân tích biệt thức tuyến tính*, principal component analysis: *phân tích thành phần chính*, outlier: *điểm ngoại biên*, feature selection: *lựa chọn đặc trưng*, feature abstraction: *trích yếu đặc trưng*, sequential forward search: *tìm kiếm tiến tuần tự*, sequential backward search: *tìm kiếm lùi tuần tự*, sequential floating forward search: *tìm kiếm tiến tuần tự di động*, training set: *tập huấn luyện*, test set: *tập thử*, confusion matrix: *ma trận đúng sai*, true positive: *dương đúng*, false positive: *dương sai*, true negative: *âm đúng*, false negative: *âm sai*, cross-validation: *kiểm tra chéo*, random sub-sampling: *lấy mẫu ngẫu nhiên*, k-fold cross-validation: *kiểm tra chéo k-phần*, leave-one-out cross-validataion: *kiểm tra chéo một mẫu thử,* sampling with replacement: *lấy mẫu được phép trùng lắp*